

Reviewer Pack — Minimal Local Index and Communication Complexity Rank Variance...

Entry 55bb82316e2c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 23:18:05 UTC

Minimal Local Index and Communication Complexity Rank Variance

Entry ID: 55bb82316e2c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 23:18:05 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Local Systems) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every k -regular graph G , the minimal local index ($\text{mli}(G)$) of its associated hyperplane arrangement is linearly correlated with the variance in communication complexity rank across all subgraphs H of G , such that $\text{mli}(G) = \Omega(\text{Var}(\text{Rank}(H)))$

Rationale (proposer's reasoning):

Algebraic topology's tools for studying local systems could provide new insights into the geometry of communication complexity. Local index measures the complexity of a system, while communication complexity rank variance captures the variability in complexity across different subgraphs. A lower bound on this correlation suggests that local index is a fundamental parameter influencing communication complexity.

Taxonomy category: local_systems_in_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 354de1eae633b6e7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a k -regular graph G with $n \leq 40$ variables, if the minimal local index $\text{mli}(G)$ is at least 10 times greater than the variance in communication complexity rank across all subgraphs H of G ($\text{mli}(G) \geq 10 * \text{Var}(\text{Rank}(H))$), then the conjecture is supported. If any seed produces a ratio less than 10 or a null variance, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local index" AND "communication complexity rank variance" - "hyperplane arrangement" IN ALGTOP AND matrix rank IN COMPLEXITYTHEORY - "subgraph H" OF graph G AND $\Omega(\text{Var}(\text{Rank}(H)))$

Top relevant hits considered: - [http://arxiv.org/abs/math/0511717v2] The Face Semigroup Algebra of a Hyperplane Arrangement - [http://arxiv.org/abs/0802.0878v3] Jumping numbers of hyperplane arrangements - [http://arxiv.org/abs/math/0605643v2] Generic section of a hyperplane arrangement and twisted Hurewicz maps - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^\pm \rightarrow D(\rightarrow K_S^0 h'^+ h'^-) h^\pm$ dec - [http://arxiv.org/abs/2604.05712v1] Precise measurement of the CKM angle γ with a novel approach - [http://arxiv.org/abs/2410.02421v2] Search for lepton number violating decays of $D_s^+ \rightarrow h^- h^0 e^+ e^+$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_k_regular_graph(n, k):
    if (n * k) % 2 != 0:
        raise ValueError("Invalid parameters for generating a d-regular graph")

    adj_list = [[] for _ in range(n)]
    degree_count = [0] * n

    while any(d < k for d in degree_count):
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)

        if u == v or v in adj_list[u]:
            continue

        adj_list[u].append(v)
        adj_list[v].append(u)
        degree_count[u] += 1
        degree_count[v] += 1

    return adj_list

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
```

```

rank = 0

for col in range(cols):
    pivot_row = -1
    for row in range(rank, rows):
        if matrix[row][col] != 0:
            pivot_row = row
            break

    if pivot_row == -1:
        continue

    matrix[pivot_row], matrix[rank] = matrix[rank], matrix[pivot_row]

    for r in range(rows):
        if r != rank and matrix[r][col] != 0:
            factor = matrix[r][col] / matrix[rank][col]
            for c in range(cols):
                matrix[r][c] -= factor * matrix[rank][c]

    rank += 1

return rank

def communication_complexity_rank(graph, subset):
    subgraph = [graph[i] for i in subset if any(j in subset for j in graph[i])]
    matrix = [[0] * len(subgraph) for _ in range(len(subgraph))]

    for i in range(len(subgraph)):
        for j in range(i + 1, len(subgraph)):
            if subgraph[j] and j in subgraph[i]:
                matrix[i][j] = 1
                matrix[j][i] = 1

    return gaussian_elimination(matrix)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        k = min(2 * (n - 1), n // 2) # Ensure the graph is regular
        graph = generate_k_regular_graph(n, k)

        total_rank = 0
        for subset_size in range(1, n + 1):
            subsets = list(itertools.combinations(range(n), subset_size))
            for subset in subsets:
                rank = communication_complexity_rank(graph, subset)
                if rank == 0:
                    continue
                total_rank += rank

    mli = len(graph) * (len(graph) - 1) / 2

```

```

    variance = total_rank / n_values.count(n) / n_values.count(n) if n_values.count(n) > 0 else 0

    results.append({
        "n": n,
        "mli": mli,
        "variance": variance
    })

mean_mli = sum(result["mli"] for result in results) / len(results)
mean_variance = sum(result["variance"] for result in results) / len(results)

conjecture_holds = all(result["mli"] >= 10 * result["variance"] for result in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Ratio of mli to variance",
    "metric_value": mean_mli / mean_variance,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{trial_result}}}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6d707246.py", line 127, in <module>

```

trial_result = run_trial(seed)
                ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6d707246.py", line 91, in run_trial
    rank = communication_complexity_rank(graph, subset)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6d707246.py", line 75, in communication
    return gaussian_elimination(matrix)
           ^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6d707246.py", line 40, in gaussian_elim
    rows, cols = len(matrix), len(matrix[0])
                                ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify the conjecture's support or falsification based on the pre-registered criteria. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15041
2	preregistration	ollama_remote	glm4:latest	0	0	9748
3	novelty	ollama_remote	glm4:latest	0	0	8534
4	novelty	ollama_remote	glm4:latest	0	0	14795
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33470
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12325
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13485
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13942
9	judge	ollama_remote	glm4:latest	0	0	11906

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 133246 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/55bb82316e2c.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/55bb82316e2c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/55bb82316e2c.tar.gz` (if generated)